

SHETH L.U.J AND SIR M.V COLLEGE  
PRACTICAL NO 2  
SUBJECT - Cyber & Information Security

AIM- Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI CODE -

```
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None

p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))

n = p * q
phi = (p - 1) * (q - 1)

e = int(input("Enter public exponent (e): "))

while gcd(e, phi) != 1:
    print("e must be coprime with phi.")
    e = int(input("Enter another value for e: "))

d = mod_inverse(e, phi)

print("\nPublic Key: ({}, {}".format(e, n))
print("Private Key: ({}, {}".format(d, n))

msg = int(input("\nEnter message (integer less than {}): ".format(n)))

cipher = pow(msg, e, n)
print("Encrypted Message:", cipher)

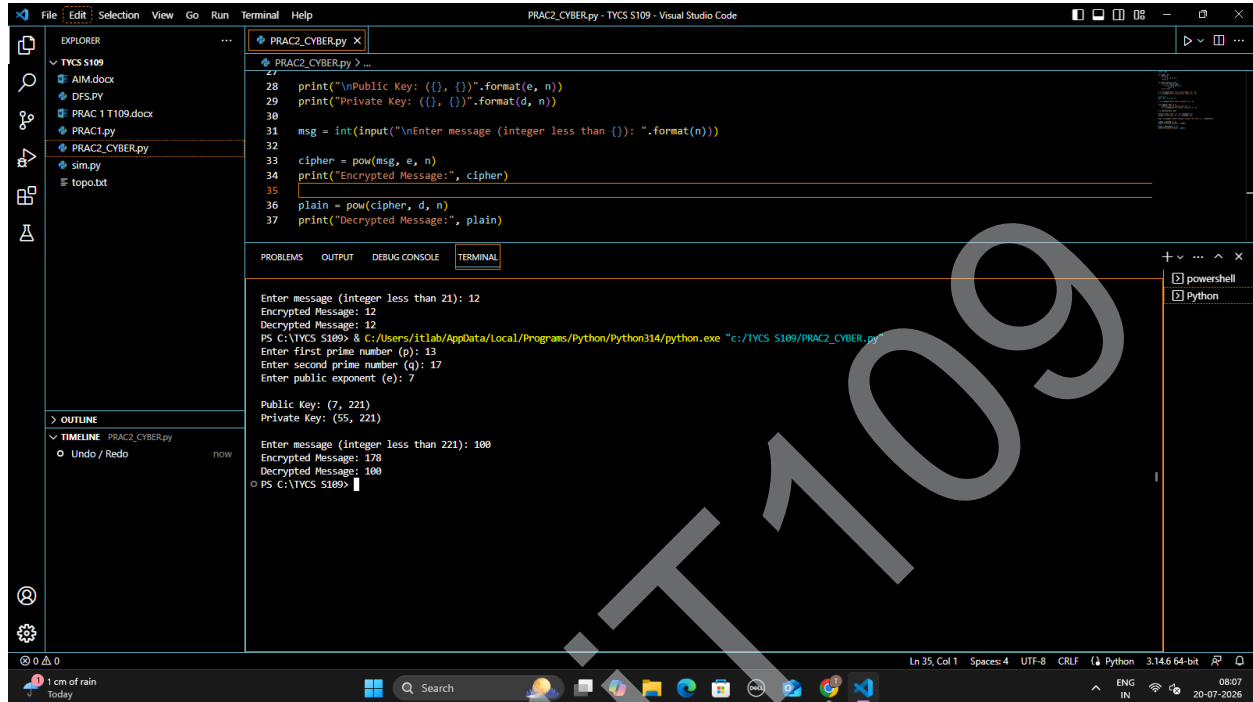
plain = pow(cipher, d, n)
```

NAME - UNNATI RATHOD  
ROLL NO - T109

SHETH L.U.J AND SIR M.V COLLEGE  
PRACTICAL NO 2  
SUBJECT - Cyber & Information Security

```
print("Decrypted Message:", plain)
```

OUTPUT -



```
28 print("\nPublic Key: ({}, {})".format(e, n))
29 print("Private Key: ({}, {})".format(d, n))
30
31 msg = int(input("\nEnter message (integer less than {}): ".format(n)))
32
33 cipher = pow(msg, e, n)
34 print("Encrypted Message:", cipher)
35
36 plain = pow(cipher, d, n)
37 print("Decrypted Message:", plain)
```

Enter message (integer less than 21): 12  
Encrypted Message: 12  
Decrypted Message: 12  
PS C:\TYCS S109> & C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.exe -c "c:/TYCS S109/PRACT2\_CYBER.py"  
Enter first prime number (p): 13  
Enter second prime number (q): 17  
Enter public exponent (e): 7  
Public Key: (7, 221)  
Private Key: (55, 221)  
Enter message (integer less than 221): 100  
Encrypted Message: 178  
Decrypted Message: 100  
PS C:\TYCS S109>

GUI CODE -

```
import tkinter as tk
from tkinter import messagebox
import math
```

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

```
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None
```

```
def rsa():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())
        msg = int(entry_msg.get())
```

NAME - UNNATI RATHOD  
ROLL NO - T109

SHETH L.U.J AND SIR M.V COLLEGE  
PRACTICAL NO 2  
SUBJECT - Cyber & Information Security

```
n = p * q
phi = (p - 1) * (q - 1)

if gcd(e, phi) != 1:
    messagebox.showerror("Error", "e must be coprime with  $\phi(n)$ .")
    return

d = mod_inverse(e, phi)

if d is None:
    messagebox.showerror("Error", "Invalid public exponent.")
    return

if msg >= n:
    messagebox.showerror("Error", "Message must be less than n.")
    return

cipher = pow(msg, e, n)
plain = pow(cipher, d, n)

lbl_public.config(text=f"Public Key: ({e}, {n})")
lbl_private.config(text=f"Private Key: ({d}, {n})")
lbl_cipher.config(text=f"Encrypted Message: {cipher}")
lbl_plain.config(text=f"Decrypted Message: {plain}")

except ValueError:
    messagebox.showerror("Error", "Please enter valid integers.")

root = tk.Tk()
root.title("RSA Encryption and Decryption")
root.geometry("450x450")

tk.Label(root, text="First Prime Number (p)").pack()
entry_p = tk.Entry(root)
entry_p.pack()

tk.Label(root, text="Second Prime Number (q)").pack()
entry_q = tk.Entry(root)
entry_q.pack()

tk.Label(root, text="Public Exponent (e)").pack()
entry_e = tk.Entry(root)

NAME - UNNATI RATHOD
ROLL NO - T109
```

SHETH L.U.J AND SIR M.V COLLEGE  
PRACTICAL NO 2  
SUBJECT - Cyber & Information Security

```
entry_e.pack()

tk.Label(root, text="Message (Integer)").pack()
entry_msg = tk.Entry(root)
entry_msg.pack()

tk.Button(root, text="Encrypt & Decrypt", command=rsa).pack(pady=10)

lbl_public = tk.Label(root, text="")
lbl_public.pack()

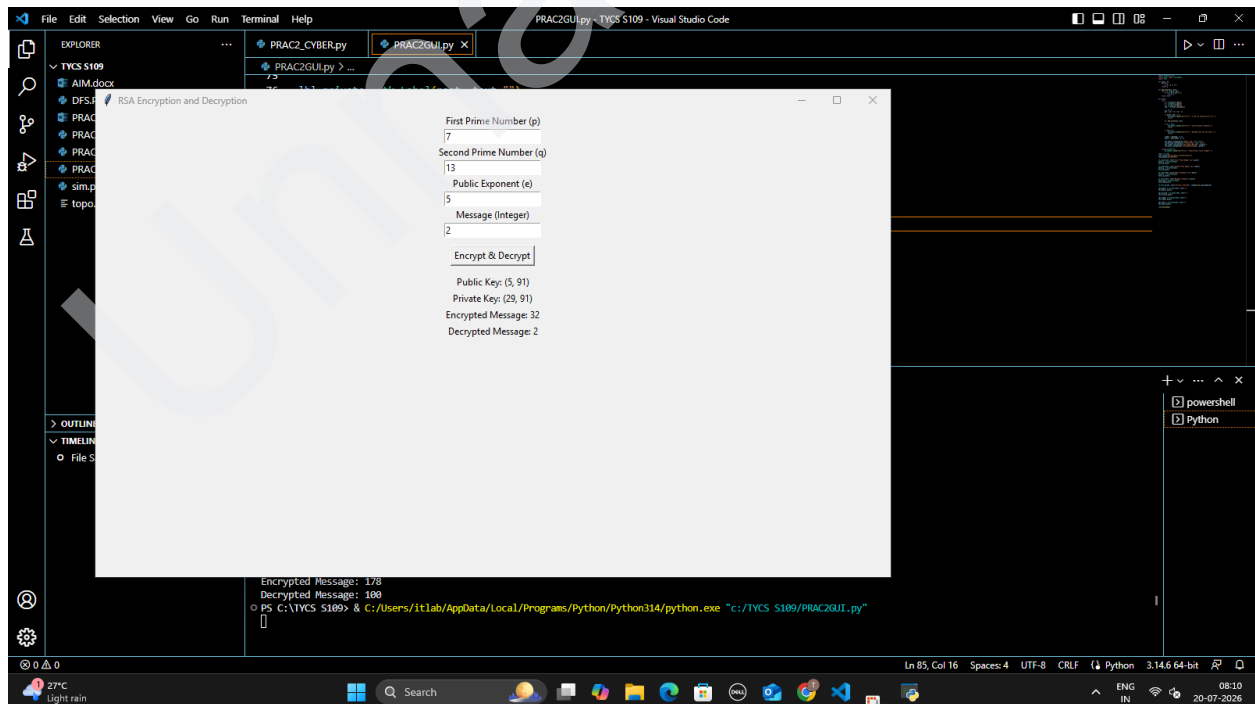
lbl_private = tk.Label(root, text="")
lbl_private.pack()

lbl_cipher = tk.Label(root, text="")
lbl_cipher.pack()

lbl_plain = tk.Label(root, text="")
lbl_plain.pack()

root.mainloop()
```

OUTPUT -



NAME - UNNATI RATHOD  
ROLL NO - T109